AARIA

Hands-Free WhatsApp Voice Messenger

Complete Architecture & Product Summary

Version 1.0 · 2026

1. What is Aaria

Aaria is an Android-based hands-free AI assistant that creates a complete voice layer on top of WhatsApp. Incoming text messages are converted to voice and read aloud through earbuds. You reply by speaking naturally — your voice is transcribed to text and sent back — without ever touching your phone.

Hinglish (Hindi + English code-switching) is a first-class language in Aaria, not an afterthought. Every layer of the pipeline is designed to handle the fluid, informal nature of real Indian WhatsApp conversations.

The immediate product is a hands-free WhatsApp voice messenger. The long-term vision is a full hands-free AI assistant. WhatsApp voice messaging is Feature 1.

2. Platform

Android only. iOS is not feasible due to platform sandboxing restrictions.

Initially distributed as a sideloaded APK — not Play Store — to avoid policy restrictions during development and testing phases.

3. System Architecture Overview

Aaria has two primary pipelines — Incoming and Outgoing — supported by a persistent Core Services layer running in the background.

INCOMING PIPELINE	OUTGOING PIPELINE
↓ WhatsApp Notification Arrives	↓ Wake Word Detected — "Hey Aaria"
↓ NotificationListenerService	↓ Audio Capture — MediaRecorder

↓ Message Queue Manager	↓ Audio Cleaning — Noise Suppression
↓ Text Intelligence Layer	↓ Dual Stop Detection — Silence / "Done"
↓ TTS Engine — Android / Azure / AI Bharat	↓ Whisper STT — API or On-Device
↓ Audio Output Manager	↓ Text Intelligence Layer (Outgoing)
↓ You Hear the Message	↓ Confirmation Read-back — 3s Window
↓ Wake Word Engine Activates	↓ RemoteInput Send → WhatsApp Delivers
CORE SERVICES LAYER — Mode Manager · Queue Manager · Audio Focus Manager · Wake Word Engine · Language Engine · Settings Store	

4. Incoming Pipeline — Detailed Architecture

Stage 1 — NotificationListenerService

The official Android API for reading notifications. Captures every notification and filters strictly by WhatsApp package names — com.whatsapp and com.whatsapp.w4b. All other notifications are completely ignored and never stored.

```
WhatsApp sends notification to Android system
  ↓
onNotificationPosted() fires
  ↓
Package filter: com.whatsapp / com.whatsapp.w4b only
  ↓
Extract: sender name · message text · RemoteInput action
  ↓
Build MessageObject { id, sender, text, isGroup, remoteInputAction, timestamp }
  ↓
Push to Message Queue
```

> ⚠ **Critical Rule**
> RemoteInput action must be stored BEFORE anything else happens.
> If the user opens WhatsApp manually, the notification dismisses and the action expires.
> Aaria detects expiry and notifies: "Reply window expired, open WhatsApp manually."

Known Hard Limitations

- Long messages are truncated to notification preview length — cannot be worked around

- Media messages (photos, voice notes, videos) show placeholders only — actual media inaccessible

Stage 2 — Message Queue Manager

Manages ordered queue of MessageObjects and handles all multi-message scenarios.

```
New MessageObject enters queue
  ↓
Is TTS currently reading?  YES → hold in queue
                           NO  → process immediately
  ↓
Count pending messages
  1 message  → read directly
  2+ messages → announce: "N messages from [names]. Say a name or say later."
  ↓
Contact name voice detection active
  Name detected  → pull message → Text Intelligence Layer
  "Later"        → queue holds silently
  No response 10s → queue holds silently
```

Stage 3 — Text Intelligence Layer (Incoming)

The most complex layer. Runs entirely on-device. Zero cost. Zero network calls. All processing happens in sequence before any TTS call is made.

01

Emoji & Media Processing

- □ → "shared a photo"
- □ → "sent a voice message"
- □ → skip or "laughing emoji"
- URLs → "shared a link"

02

Abbreviation Expansion

- yr → yaar
- h (sentence end) → hai
- k → ok
- m → main / mein
- srsly → seriously
- tbh → to be honest
- brb → be right back
- bc → because (context-based)
- Custom user-defined expansions

03

Language Detection — fastText On-Device

- Model: fastText via TFLite
- Size: <2MB | Inference: <5ms
- Trained on: HuggingFace Hinglish datasets + synthetic WhatsApp data
- Output: Language Profile Object { primary, hindi_ratio, english_ratio, script, devanagari_present }
- Profile travels with message through all downstream layers

04

Word-Level Language Tagging

- Each word tagged individually as Hindi or English
- "bhai seriously bohot bura hua" → [HI] [EN] [HI] [HI] [HI]
- Based on fastText word scoring + vocabulary lookup tables

05

Transliteration — indic-transliteration

- Open source library, runs on-device
- Hindi-tagged words: Roman → Devanagari
- "bhai" → "□□□" | "bohot" → "□□□□" | "bura" → "□□□□"
- English words left untouched in Roman script

06

SSML Construction

- Builds Speech Synthesis Markup Language with per-word language tags
- <lang xml:lang="hi-IN"> for Hindi words
- <lang xml:lang="en-IN"> for English words
- Enables natural voice switching within a single sentence

Stage 4 — TTS Engine

Three-tier engine selection — all free options. Swap engines without changing any other layer.

Tier	Engine	Quality	Cost
Primary	Android Built-in TTS with SSML (hi-IN + en-IN voices)	Good	Free
Upgrade	Azure Free Tier — hi-IN-SwaraNeural + en-IN-NeerjaNeural	Better	Free
Advanced	AI Bharat TTS self-hosted on Oracle Cloud Free Tier	Best	Free

Voice Cloning — Personal Voice Mode

Aaria supports cloning a specific person's voice for TTS output using their WhatsApp voice notes as training samples.

- Minimum samples: 1–3 minutes clean audio (functional clone)
- Recommended: 20–30 minutes of natural conversational audio from WhatsApp voice notes
- Tool: ElevenLabs (free tier, 1 custom voice) or Coqui XTTS / OpenVoice (free, self-hosted)
- In Personal Voice Mode: full Romanised Hinglish text passed to cloned voice model directly — no SSML splitting needed, model handles code-switching naturally
- Consent: voice cloning a person's voice requires their explicit knowledge and agreement

Stage 5 — Audio Output Manager

- Requests AudioFocusRequest before any TTS playback, releases after
- Detects active phone call — suppresses TTS, queues message for after call
- Routes to Bluetooth earbuds if connected, wired earphones, or speaker based on preference
- Ducks background music to 20% while reading, restores after
- Group message prefix: "Message from [name] in [group]: [message]"
- TTS queue — multiple messages read sequentially, never simultaneously

5. Outgoing Pipeline — Detailed Architecture

Stage 1 — Wake Word Engine

Picovoice Porcupine runs on-device. No network call. Minimal battery usage.

Battery Optimisation

Wake word listener is NOT active 24/7.
Activates ONLY after a message arrives and TTS finishes reading.
Deactivates after reply window closes (sent, cancelled, or timed out).
Effective battery cost is near zero.

Wake Word States

- INACTIVE — default, zero battery draw
- LISTENING_FOR_REPLY — post-TTS window, listening for "Hey Aaria" and contact names
- LISTENING_FOR_STOP — during recording, listening for "Done"
- LISTENING_FOR_CANCEL — during read-back window, listening for "cancel"

Extended Functionality — Voice Commands

- "Hey Aaria, driving mode" → switch to Driving Mode
- "Hey Aaria, focus on [name]" → activate Focus Mode
- "Hey Aaria, exit focus" → deactivate Focus Mode

Stage 2 — Audio Capture & Dual Stop Detection

MediaRecorder starts on wake word confirmation. Two stop mechanisms run in parallel — first to fire wins.

Silence Detection (VAD)	Stop Word Detection
Tool: Silero VAD (on-device)	Tool: Porcupine second keyword
Trigger: 1.5 seconds of silence	Trigger: User says "Done"
Best for: quiet environments	Best for: noisy environments
Feels invisible — user just stops talking	Reliable fallback regardless of noise

The stop word "Done" is stripped from the audio clip before Whisper receives it — it never appears in the transcription.

Stage 3 — Audio Cleaning

- Stop word clip — identify "Done" timestamp, clip audio at that point
- Background noise suppression
- Echo cancellation
- Silence stripped from start and end of clip
- Clean audio file passed to STT engine

Stage 4 — Speech to Text (Three-Tier Architecture)

STT uses a three-tier fallback system. Whisper API is the primary engine. On-device whisper.cpp is the offline fallback. Android SpeechRecognizer is the last resort.

Tier	Tool	Hinglish Quality	Network	Cost
Primary	OpenAI Whisper API (whisper-1)	Excellent	Required	~$1/month
Offline Fallback	whisper.cpp on-device (base model, 74MB)	Good	None	Free

Tier	Tool	Hinglish Quality	Network	Cost
Last Resort	Android SpeechRecognizer offline mode	Acceptable	None	Free

Whisper API Configuration

- Model: whisper-1
- Prompt parameter: "Hinglish conversation, Roman script Hindi mixed with English, casual WhatsApp style"
- This prime significantly improves Hinglish output consistency and reduces over-correction to pure English
- Cost: $0.006/min → 10 second reply = $0.001 → 1000 replies/month ≈ $1

Stage 5 — Text Intelligence Layer (Outgoing)

Post-processes Whisper output before read-back. Preserves personal voice — does not change meaning.

01 **Stop Word Safety Strip**
Secondary check — did "Done" or "Send" leak into transcription? Strip if found at end.

02 **Devanagari → Roman Conversion**
Whisper sometimes returns Hindi words in Devanagari. Recipients expect Roman script. Convert back.

03 **Filler Word Removal**
"um", "uh", "you know", "like" stripped from transcription.

04 **Punctuation & Capitalisation Fix**
Whisper sometimes returns all lowercase or misses punctuation. Light correction applied.

05 **Light Normalisation**
Minimal — preserve personal voice. Do not over-correct Hinglish. Do not change meaning.

Stage 6 — Confirmation & Send

```
Final message text ready
  ↓
```

```
TTS reads back: "Sending: [message text]"
  ↓
3 second auto-send window opens
  Porcupine listening for "cancel"
  ↓
"Cancel" detected → abort send → return to queue
No cancellation → proceed
  ↓
Retrieve stored RemoteInput action for MessageObject
  Action still valid? YES → fire with final text
                      NO  → notify: "Reply window expired, open WhatsApp
manually"
  ↓
Confirmation chime in earbuds → TTS: "Sent."
  ↓
Queue manager: any remaining messages? Announce or close window.
```

6. Core Services Layer

Mode Manager

Mode	Behaviour
Driving Mode	Full hands-free. Wake word active during reply windows. All messages announced and queued. Stop word essential. Read-back before send. Zero distraction optimised.
Focus Mode	Single contact filter. Only focused contact messages read aloud. Others silently queued. Activated: "Hey Aaria, focus on [name]". Deactivated: "Hey Aaria, exit focus".
Silent Mode	Nothing read aloud. All messages queue silently. Respects system Do Not Disturb. Manual check only.
Normal Mode	Passive background. No auto-reading. Hands-free reply available but not auto-triggered. Default state.

Mode switching via voice command through wake word at any time, or via quick settings tile for single-tap switching.

Language Engine

Shared service used by both pipelines. Runs entirely on-device. No network. Free.

- fastText classifier via TFLite — <2MB, <5ms inference
- Hindi vocabulary lookup table
- English vocabulary lookup table
- Abbreviation expansion dictionary — user-editable

- indic-transliteration library — Roman Hindi ↔ Devanagari
- SSML builder — per-word language tag construction
- Devanagari to Roman converter — for outgoing Whisper output cleanup

7. Complete Hands-Free Experience

End-to-End Flow — Noisy Environment, Multiple Messages

01	Soft chime plays in earbuds
02	Aaria: "3 messages — Rahul, Mom, Team group. Say a name to reply or say later."
03	Wake word listener active, also listening for contact names
04	You say: "Rahul"
05	Aaria reads Rahul's message naturally in Hinglish — proper Hindi pronunciation via SSML
06	Chime — reply window open. Silence detection + stop word both active simultaneously
07	You say: "Hey Aaria... haan yaar 20 minutes mein aata hoon... Done"
08	"Done" fires → audio cleaned and clipped → sent to Whisper
09	Whisper returns: "haan yaar 20 minutes mein aata hoon"
10	Text intelligence: stop word check, Devanagari check, filler removal, punctuation fix
11	Aaria reads back: "Sending: haan yaar 20 minutes mein aata hoon"
12	3 second window — no cancellation → auto-sends via RemoteInput
13	Aaria: "Sent. 2 messages remaining — Mom, Team group. Say a name or say later."

Result

Zero screen touches. Zero button presses. Natural conversation through your ears.

8. Complete Tech Stack

Component	Tool	Cost
Language	Kotlin	Free
Message Capture	NotificationListenerService	Free
Reply Sending	RemoteInput API	Free
Language Detection	fastText on-device via TFLite (<2MB)	Free
Abbreviation Expansion	Local Hinglish dictionary	Free
Word-Level Tagging	On-device classifier + vocab lookup	Free
Transliteration	indic-transliteration library	Free
SSML Construction	On-device	Free
TTS Primary	Android Built-in TTS	Free
TTS Upgrade	Azure Free Tier — hi-IN-SwaraNeural	Free (<500k chars/month)
TTS Advanced	AI Bharat on Oracle Cloud Free Tier	Free
Voice Cloning	ElevenLabs (1 free custom voice) or Coqui XTTS	Free tier
Wake Word + Stop Word	Picovoice Porcupine	Free tier
Silence Detection	Silero VAD (on-device)	Free
Audio Cleaning	On-device processing	Free
STT Primary	OpenAI Whisper API (whisper-1)	~$1/month
STT Offline Fallback	whisper.cpp on-device (base model)	Free
STT Last Resort	Android SpeechRecognizer offline	Free
HTTP Client	OkHttp + Retrofit	Free
Background Service	Android Foreground Service	Free

Total Monthly Cost

Approximately $1/month at personal use scale. The only paid component is OpenAI Whisper API for STT — every other component is free.

9. Permissions Required

Permission	Purpose
BIND_NOTIFICATION_LISTENER_SERVICE	Read WhatsApp notifications
RECORD_AUDIO	Wake word, stop word, and voice recording
INTERNET	Whisper API and optional Azure TTS
SYSTEM_ALERT_WINDOW	Floating overlay bubble UI
FOREGROUND_SERVICE	Keep notification listener alive in background
VIBRATE	Haptic feedback on send confirmation

10. Known Concerns & Mitigations

Concern	Mitigation
OEM battery optimisation killing service	Onboarding flow with manufacturer-specific battery whitelist instructions (Xiaomi, Realme, Samsung)
Notification permission friction	Clear onboarding explaining exactly what is and is not read by Aaria
OTP and sensitive notification exposure	Strict package whitelist — only com.whatsapp and com.whatsapp.w4b ever processed
Wake word false positives	Porcupine sensitivity tuning. Listener not active 24/7 which significantly reduces exposure
RemoteInput expiry race condition	Expiry detection with clear user notification. No silent failures
No internet connectivity	Three-tier STT fallback. TTS always works offline via Android built-in

Concern	Mitigation
Audio focus conflicts	Proper AudioFocusRequest lifecycle. Pause during calls, play nicely with music apps
Long messages truncated	Hard limitation of NotificationListener. Communicated clearly in onboarding
Media messages inaccessible	Hard limitation. Placeholder description read: "shared a photo"
Multi-device WhatsApp	Low priority edge case monitored, not solved in v1

11. Development Phases

Phase	Focus	Deliverable
1	Message Capture	NotificationListenerService working. WhatsApp package filter. Extract sender, text, store RemoteInput action. Prove data capture pipeline works.
2	Voice Output	Android TTS reading messages aloud. Message queuing. Group prefixing. Audio focus management. Get the listening experience right.
3	Language Intelligence	fastText on-device classifier. Abbreviation expander. Word-level tagger. indic-transliteration. SSML construction. Natural Hinglish reading working.
4	Wake & Stop Word	Porcupine integration for "Hey Aaria" and "Done". MediaRecorder pipeline. Dual stop detection tested in quiet and noisy environments.
5	Full Reply Loop	Whisper STT + whisper.cpp fallback. Audio cleaning. Outgoing text intelligence. Confirmation read-back. RemoteInput send. End-to-end loop working.
6	Multi-Message Queue	Queue management. TTS sender announcements. Contact name voice selection. Queue behaviour testing with simultaneous messages.
7	Modes	Driving, Focus, Silent, Normal modes. Voice command switching. Quick settings tile. Focus Mode contact filtering and background queue.
8	Polish & Edge Cases	Bluetooth integration. Earbud button fallback. Battery optimisation onboarding. Settings screen. Voice cloning integration. Stability testing.

12. Open Questions for Future Phases

Intelligence Layer — Not In Scope for v1

Smart reply suggestions based on conversation context.
Tone adjustment — more formal or casual based on contact.
Learning personal texting style and abbreviation patterns over time.
Cross-app expansion beyond WhatsApp.
The layered architecture accommodates all of these cleanly without disrupting what is built in Phases 1–8.